

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 02 -

AGENDAMENTO AVANÇADO E COMPORTAMENTO DE NODES

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	19
O QUE VOCÊ VIU NESTA AULA?	19
REFERÊNCIAS.....	23

EMANIP

O QUE VEM POR AÍ?

Nesta seção, vamos mergulhar em técnicas avançadas para dominar o agendamento de Pods e a resiliência dos nós em seu cluster Kubernetes. Responderemos a perguntas cruciais: como garantir que uma aplicação rode apenas em máquinas com hardware específico, como GPUs? E como impedir que um nó sobrecarregado por falta de memória ou disco cause uma falha em cascata?

Primeiro, você aprenderá a usar Taints e Tolerations para "repelir" Pods de nós inadequados, garantindo o isolamento de recursos críticos. Em seguida, exploraremos a Node Affinity para "atrair" suas aplicações para os nós ideais, otimizando performance e custos.

Por fim, desvendaremos o mecanismo de Eviction (remoção sob pressão), a estratégia de autoproteção do Kubernetes que remove Pods de forma inteligente para salvar um nó antes que ele falhe. Prepare-se para aprender a orquestrar suas cargas de trabalho com precisão e a construir um cluster mais robusto e eficiente.

HANDS ON

Nesta seção hands on, construiremos um mini-agendador em Rust para simular a lógica de decisão do Kubernetes. O código modela Nós com Taints e Pods com Tolerations e Node Affinity, implementando as regras de compatibilidade.

Em nosso cenário, você verá um Pod de Machine Learning ser corretamente agendado em um nó com GPU, pois ele tolera o Taint específico e satisfaz a Affinity do nó. Simultaneamente, outros Pods são repelidos por esse Taint e alocados em nós de uso geral.

Este exercício prático demonstrará visualmente como essas regras funcionam em conjunto para garantir que cada aplicação rode exatamente onde deveria.

SAIBA MAIS

Vamos agora dissecar com mais profundidade cada um dos tópicos – taints/tolerations, node affinity e evicções – incorporando considerações teóricas (incluindo matemática e algoritmos de ciência da computação) e práticas de implementação em contêineres. Lembre-se de que nosso foco é sempre pensar como essas funcionalidades do Kubernetes resolvem problemas de gerenciamento de *clusters* em escala, garantindo uso eficiente de recursos e alta disponibilidade das aplicações.

Taints e Tolerations: Repelindo Pods de Nós Inadequados

No Kubernetes, **taints** e **tolerations** trabalham em conjunto para impedir que Pods sejam agendados em nós “inadequados” – a não ser que expressem explicitamente que toleram aquela condição. Em outras palavras: *taints* são como sinais nos nós dizendo “não coloque Pods aqui, a menos que eles saibam lidar com X”, e *tolerations* são a marca no Pod dizendo “eu sei lidar com X, posso ir naquele nó”. Essa mecânica é fundamental para **isolar cargas de trabalho** ou **reservar nós exclusivos** para certos propósitos.

Sintaxe e Efeitos: um taint é composto por uma chave, um valor e um efeito. Por exemplo: `kubectl taint nodes node1 dedicacao=BD:NoSchedule` marcaria o nó *node1* com a chave *dedicacao*, valor *BD* (banco de dados) e efeito *NoSchedule*. Esse efeito significa que nenhum Pod deve ser agendado ali a não ser que tolere essa marca. Os efeitos podem ser:

- **NoSchedule** impede agendamento de novos Pods não tolerantes (pods já rodando não são expulsos).
- **PreferNoSchedule**: indica uma preferência para não agendar (o scheduler tentará evitar, mas não é absoluto).
- **NoExecute**: além de impedir novos agendamentos, **ejetará imediatamente qualquer Pod não tolerado que já estiver rodando no nó**. É útil, por exemplo, para esvaziar um nó para manutenção – você adiciona um taint *NoExecute* e todos os Pods sem toleration são *evictados* (removidos) rapidamente.

Tolerations nos Pods: do lado do Pod, especificamos tolerations geralmente no spec do Deployment/Pod. Uma toleration típica pode ser:

```
tolerations:
- key: "dedicacao"
  operator: "Equal"
  value: "BD"
  effect: "NoSchedule"
```

Isso “casa” com o taint do exemplo acima (`dedicacao=BD:NoSchedule`), permitindo que o Pod seja escalado naquele nó. Podemos também usar `operator: "Exists"` para tolerar um taint independentemente do valor (basta a chave e o efeito baterem).

Tolerar não significa exigir. Um Pod com essa toleration pode ser agendado em node1, mas não será forçado para lá – ele apenas não será impedido por esse critério. O scheduler continuará avaliando outros fatores (como recursos disponíveis, afinidade, disponibilidade e configurações) para decidir qual nó é o melhor.

Tolerations permitem o agendamento, mas não o garantem.

No processo de **filtragem** de nós candidatos, o scheduler elimina todos os nós que tenham *taints* não tolerados pelo Pod. É uma etapa binária: ou o Pod tolera todos os taints do nó, ou aquele nó é descartado das opções. Essa é uma implementação direta de um conceito de **satisfação de restrições** em ciência da computação – podemos ver um nó com taints como um elemento que deve satisfazer o predicado “Pod tolera todos os meus rótulos de exclusão?” para ser considerado viável.

Se considerarmos cada taint como um conjunto de Pods que o toleram (um Pod sem toleration pertence ao conjunto vazio para aquele taint), então a filtragem checka se o Pod pertence à interseção de conjuntos de tolerância para todos os taints do nó. Se não pertencer, exclusão.

Um uso clássico de taints é **dedicar nós para certos componentes**. Kubernetes por padrão marcava os nós mestre com `node-role.kubernetes.io/control-plane:NoSchedule`, de modo que apenas componentes

críticos (que já têm tolerations apropriadas) rodem neles, mantendo workloads de usuário afastados.

É comum fazer esta aplicação para nós com recursos caros, como GPUs, colocando-se a configuração apropriada (por exemplo, `nvidia.com/gpu=true:NoSchedule`, caso se esteja utilizando infraestrutura Nvidia) em nós com GPU e então só os Pods de ML ou CUDA que tiverem a toleration correspondente poderão ir pra lá.

Sem isso, um Pod comum poderia ser escalado num nó de GPU e nem usar a placa, enquanto um Pod de ML poderia cair num nó sem GPU e falhar. Os taints evitam esses cenários, atuando como “cordões de isolamento”. Vale notar que *DaemonSets* importantes (logs, métricas) costumam vir com toleration do tipo `Exists` sem chave para rodar em todos os nós, inclusive os taintados, garantindo que mesmo nós dedicados ainda executem Pods de infraestrutura essenciais.

Um ponto sutil é que taints só **repelem** Pods, eles não atraem Pods específicos para si. Se marcamos metade dos nós com `env=dev:NoSchedule` e a outra metade com `env=prod:NoSchedule`, e temos um Pod de ambiente dev que tolera `env=dev`, esse Pod *pode* ir pros nós dev (pois tolera) **mas também pode acabar agendado em um nó sem taint nenhum, ou seja, um nó que não era “dev”**.

A toleration não obriga o Pod a ir para o nó taintado, ela apenas o habilita. Por isso, a melhor prática de isolamento forte combina taints/tolerations **com Node Affinity**, para cobrir os dois lados. Discutiremos isso adiante, mas já fica claro: se você quer **garantir** que um certo Pod só rode em certo grupo de nós, apenas tolerations não bastam – você tolera o nó “especial” e define afinidade dizendo que ele só deve ir lá, evitando cair em nó genérico por engano. Um detalhe avançado é que tolerations para *NoExecute* taints podem especificar um tempo de tolerância. Exemplo:

```
tolerations:
- key: "dedicacao"
  effect: "NoExecute"
  tolerationSeconds: 600
```

Isso significaria que, se durante a execução do Pod um taint `dedicacao` aparecer no nó, o Pod tolerará por 600 segundos antes de ser evicto (removido). Sem esse campo, tolerar `NoExecute` significa tolerar indefinidamente. Essa opção é útil para cenários como tolerar temporariamente um nó não-preparado, facilitando a alocação de recursos em situações de falhas isoladas ou instanciamento ou agendamento personalizado e adaptado às necessidades do sistema. Por padrão, Pods toleram `node.kubernetes.io/not-ready:NoExecute` por 300s (5 minutos), permitindo janelas curtas de desconexão do nó antes de expulsar os Pods dele.

Assim sendo, taints e tolerations implementam no Kubernetes uma espécie de **política de afinidade negativa (anti-afinidade) por nó**, mas de forma declarativa e facilmente gerenciável. Em termos matemáticos, podemos imaginar que cada Pod P e Nó N definem uma relação válida se $Taints(N) \subseteq Tolerations(P)$ (todo taint do nó é tolerado pelo Pod).

Essa verificação tem complexidade $O(\tau_{a,n} \times \tau_{o,p})$, onde $\tau_{a,n}$ são as taints por nó e $\tau_{o,p}$ são as toleration por pod, para cada par Pod-Nó. Isso significa uma complexidade algorítmica bem escalável, pois tem uma taxa de crescimento constante com a quantidade de pods e nós, favorecendo o cálculo em larga escala para a alocação de processamento computacional. O scheduler nativo otimiza isso com índices e cache de taints comuns, mas conceitualmente é uma verificação relativamente barata comparada a outras (como checar disponibilidade de recursos, que requer comparar vetores de CPU/memória livres).

Node Affinity: Atraindo Pods para Nós Específicos

O **node affinity** (afinidade de nó) é a contraparte “positiva” das restrições de agendamento. Enquanto taints dizem “não venha para cá”, a afinidade de nó nos Pods diz “eu *quero* ir para nós que tenham característica X”. Essa característica geralmente é uma etiqueta (*label*) do nó. Labels de nó são pares chave-valor atribuídos a cada nó, geralmente indicando propriedades como zona de disponibilidade, tipo de máquina, propósito (dev/prod), presença de GPU, entre outras características que variam de caso a caso. A afinidade pega carona no mecanismo de labels para permitir dois tipos de regras:

`requiredDuringSchedulingIgnoredDuringExecution`: afinidade **obrigatória** (hard requirement). Se o nó não satisfaz, o scheduler nem agenda o Pod lá. Funciona

parecido com o antigo `nodeSelector`, mas com sintaxe mais expressiva (suporte a operadores In, NotIn,).

`preferredDuringSchedulingIgnoredDuringExecution`: afinidade **de preferência** (soft requirement). O scheduler *tenta* satisfazer, mas se não conseguir, tudo bem – o Pod pode ser agendado em um nó que não tenha aquele label, se necessário.

Os sufixos “IgnoredDuringExecution” significam que essas regras só importam no momento do scheduling. Se depois o label do nó mudar ou o ambiente mudar, o Pod já em execução não será expulso do nó por conta disso.

Existe teoricamente um `requiredDuringSchedulingRequiredDuringExecution` que obrigaria realocação se a condição deixasse de ser atendida, mas ele não foi implementado nas versões estáveis do Kubernetes.

Node affinity é parte do `.spec.affinity` do arquivo de configuração do Pod ou Deployment. Por exemplo:

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - key: "topology.kubernetes.io/zone"
              operator: In
              values: ["eastus-b", "eastus2-c"]
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
        preference:
          matchExpressions:
            - key: "node-type"
              operator: In
              values: ["spot"]
```

No exemplo hipotético acima, estamos exigindo que o nó esteja na zona `eastus-b` ou `eastus2-c`, e dizendo que **preferimos** (peso 1) que ele tenha o label `node-type=spot`. Isso poderia refletir uma política de agendamento para pods que tanto faz entre duas zonas (ambas atendem, mas não pode ser fora delas) e que idealmente rodem em nós spot (mais baratos), mas se não tiver disponível tudo bem rodar em on-demand.

O scheduler vai filtrar apenas nós das zonas permitidas (regra obrigatória) e, dentre esses, se alguns tiverem `node-type=spot`, esses receberão um `score` maior na fase de **pontuação** do scheduling. O algoritmo do scheduler soma pesos das preferências satisfeitas e escolhe o nó com maior pontuação total. Assim, nossas preferências de afinidade influenciam a decisão final.

A sintaxe de *nodeAffinity* permite combinar várias expressões e termos. As regras seguem lógica booleana: - Dentro de um mesmo `nodeSelectorTerm`, várias `matchExpressions` são combinadas com **E (AND)** – ou seja, o nó precisa satisfazer todas as expressões daquele termo (ex: região = eastus **E** tipo = spot).

Se houver múltiplos `nodeSelectorTerms` (lista de listas de expressões) dentro de *requiredDuringScheduling*, eles são combinados por **OU (OR)** – basta um dos termos completos ser satisfeito. Isso permite expressar alternativas (ex: “(região=eastus **E** spot) OU (região=westus **E** spot)” poderia ser duas cláusulas).

Para *preferredDuringScheduling*, cada preferência tem seu próprio peso e é avaliada independentemente somando a pontuação. Não é exatamente uma lógica OR/AND, e sim uma soma de pontos para cada preferência atendida.

Uma outra característica da definição de afinidades entre Pods e Nodes é que ela pode ser “negativa”, de forma a se controlar a colocação relativa entre Pods (`podAffinity` e `podAntiAffinity`). Afinidade e anti-afinidade entre Pods são mecanismos do planejador do Kubernetes pensados para posicionar uma carga de trabalho em relação a outras cargas de trabalho já existentes, e não apenas em relação a rótulos do nó.

Conceitualmente, em vez de perguntar “em que nó este Pod cabe?”, o planejador passa a responder “em que domínio topológico — nó, rack, zona ou região — este Pod deve ficar, considerando os Pods que já estão lá”. Essa relação é expressa por um seletor de rótulos que aponta para os Pods de interesse e por um `topologyKey` que define o domínio topológico a ser considerado; o resultado é uma política de co-localização (afinidade) ou de repulsão (anti-afinidade) avaliada no momento do agendamento.

A documentação oficial descreve exatamente essa semântica: as regras são expressas com seletores de rótulos, associadas a namespaces se necessário, e o domínio é dado por uma chave de rótulo presente nos nós, como

`kubernetes.io/hostname` para o nível de nó ou `topology.kubernetes.io/zone` para o nível de zona. O Kubernetes, então, verifica essas regras quando escolhe um nó para o Pod, levando em conta o estado atual do cluster e os Pods que já satisfazem o seletor naquele domínio topológico indicado pelo `topologyKey`.

Há dois regimes de avaliação dessas regras. No modo “obrigatório” (`requiredDuringSchedulingIgnoredDuringExecution`), o planejador só agenda o Pod em nós que satisfaçam a regra; no modo “preferencial” (`preferredDuringSchedulingIgnoredDuringExecution`), a regra informa a pontuação e influencia a escolha, mas não bloqueia o agendamento se não puder ser satisfeita. A comparação oficial com *topology spread constraints* sintetiza a intuição: afinidade “atrai” Pods para empacotá-los num domínio, enquanto anti-afinidade “repele” Pods desse domínio; no caso de anti-afinidade obrigatória, o efeito prático é limitar a um único Pod correspondente por domínio topológico, ao passo que a variante preferencial deixa de impor essa garantia e apenas orienta a escolha do nó.

Dois avisos operacionais são particularmente relevantes em ambientes de produção. Primeiro, o custo computacional, já que o inter-Pod (anti)afinidade requer processamento substancial no planejador e podem degradar a performance em clusters grandes; a própria documentação não recomenda seu uso em ambientes maiores do que “algumas centenas” de nós.

Essa cautela decorre do fato de que cada decisão de posicionamento precisa correlacionar o Pod candidato com o conjunto de Pods já existentes que se casam com o seletor e, então, calcular as possibilidades por domínio topológico — um espaço de busca que cresce com o número de nós e de Pods. Quando a escalabilidade do agendamento é um requisito explícito, convém medir o impacto e considerar alternativas como *topology spread constraints* para objetivos de espalhamento com limites quantitativos de *skew*.

Segundo, temos a consistência de rótulos nos nós, já que a anti-afinidade pressupõe que todos os nós possuam o rótulo correspondente ao `topologyKey`; a ausência do rótulo em parte do cluster pode produzir um comportamento inesperado, porque o planejador não consegue avaliar corretamente o domínio e, portanto, a repulsão pretendida. As duas recomendações — sobre custo e sobre rotulagem consistente — estão registradas na seção de afinidade/anti-afinidade da página “Assigning Pods to Nodes” da documentação oficial.

Há ainda uma restrição de segurança e desempenho que muitos operadores encontram na prática: por padrão, o *controle de admissão* definido por `LimitPodHardAntiAffinityTopology` limita o `topologyKey` das regras de anti-afinidade obrigatória ao rótulo de nível de nó `kubernetes.io/hostname`. O objetivo é evitar que uma regra “dura” de anti-afinidade seja aplicada a domínios muito largos, como uma zona inteira, o que poderia, por simetria, bloquear o agendamento de um grande conjunto de Pods e até causar uma forma de negação de serviço involuntária contra o cluster. Essa limitação pode ser modificada ou desativada pelos administradores, mas a sua existência explica por que muitos guias e exemplos de alta disponibilidade optam por anti-afinidade obrigatória no nível de nó e não de zona.

A utilidade prática dessas políticas emerge em dois grupos de situações: Quando o objetivo é reduzir latência e maximizar *localidade de dados* entre serviços que se comunicam intensamente — por exemplo, colocar instâncias de front-end e cache na mesma zona — a afinidade fornece um critério de co-localização que captura diretamente a relação entre os serviços, algo que políticas baseadas apenas em rótulos de nó não expressam com fidelidade.

De forma oposta, quando a meta é limitar o raio de falha e aumentar disponibilidade dispersando réplicas equivalentes, a anti-afinidade fornece uma forma declarativa de evitar co-alocação de réplicas no mesmo domínio de falha. Esses dois padrões, de “aproximar quem conversa muito” e de “separar quem não deve cair junto” são explicitamente discutidos na documentação conceitual, inclusive com a observação de que a versão obrigatória da anti-afinidade efetivamente impede múltiplos Pods correspondentes no mesmo domínio, enquanto a versão preferencial apenas influencia o *scoring* do planejador.

A comparação com *taints* e *tolerations* ajuda a decidir quando usar cada mecanismo. Taints são aplicados no nó e funcionam como um “mantenha distância”: o nó repelirá, por padrão, qualquer Pod que não tenha a tolerância compatível; tolerations, por sua vez, são declaradas no Pod e apenas permitem que o agendamento ocorra naquele nó, sem garanti-lo.

Em outras palavras, *taints/tolerations* são políticas de admissão por nó, excelentes para reservar nós a determinados tipos de cargas, como hardware especializado (GPUs), *pools* dedicados, nós de manutenção ou para controlar evacuação por problemas de saúde do nó, enquanto afinidade/anti-afinidade são

políticas relacionais entre Pods, ancoradas em rótulos de Pods e domínios topológicos de nós.

A documentação de *taints/tolerations* destaca esses papéis, enumera os efeitos (**NoSchedule**, **PreferNoSchedule**, **NoExecute**) e descreve casos em que convém combiná-los com afinidade de nó para construir nós dedicados a certos grupos de workloads ou usuários. Também registra o mecanismo de “evicção baseada em taints”, usado quando o controlador de nós adiciona automaticamente taints para condições como falta de memória ou pressão de disco, modulando a permanência de Pods via *tolerationSeconds*.

Do ponto de vista de projeto, isso conduz a uma regra de decisão clara e tecnicamente motivada. Sempre que o requisito fundamental for sobre a relação Pod↔Pod — aproximar serviços acoplados ou separar réplicas iguais por nó, zona ou outro domínio — a escolha natural recai sobre *podAffinity* ou *podAntiAffinity*, porque somente esses mecanismos consultam rótulos de Pods já em execução e tratam diretamente o domínio topológico onde essa relação deve valer.

Quando o requisito é admitir ou excluir classes inteiras de Pods em nós específicos, por política de plataforma, compliance, custo, tipos de hardware, manutenção ou saúde do nó, os *taints* são mais adequados, por operarem diretamente no espaço de estados do nó e escalarem de forma estável com o tamanho do cluster.

Para objetivos de “espalhamento uniforme com garantias quantitativas”, como manter uma diferença máxima de réplicas por zona, as *Pod Topology Spread Constraints* são frequentemente preferíveis à anti-afinidade porque expõem parâmetros como *maxSkew* e políticas de quando a restrição não pode ser satisfeita, além de contarem com valores padrão em versões recentes do *scheduler* que promovem um espalhamento saudável mesmo quando o usuário não especifica nada. A própria documentação traz uma seção de comparação onde se explica que afinidade “empacota”, anti-afinidade “repele” e que *topology spread* fornece o controle numérico do desbalanceamento desejado, inclusive com restrições padrão em *hostname* e *zone* quando nenhuma configuração é fornecida.

Finalmente, vale notar a interação com preempção e prioridade. Se um Pod pendente tem afinidade com Pods de menor prioridade já presentes num nó, a remoção desses “alvos” pela preempção tornaria impossível satisfazer a afinidade, e

o planejador, nesse caso, tende a procurar outro nó em vez de preemptar. Essa nuance reforça que afinidade/anti-afinidade são vínculos avaliados na composição do estado do cluster, e que intervenções como preempção precisam respeitar, tanto quanto possível, essas dependências declarativas entre Pods.

De forma geral, do ponto de vista teórico, **o problema geral de alocar Pods a nós com múltiplas restrições é complexo**. Podemos modelá-lo como um caso de **problema da mochila múltipla** (bin packing) **com restrições**: cada nó é uma mochila com capacidade (CPU, memória) e atributos; cada Pod tem um tamanho (requests de recursos) e requisitos (rótulos desejados, tolerations). Encontrar a alocação ótima de um conjunto de Pods em nós que minimize sobras de recursos e respeite todas as restrições é NP-difícil (NP-hard) – semelhante ao clássico problema de *bin packing*, que não tem solução ótima eficiente em geral.

O scheduler do Kubernetes, portanto, usa heurísticas e aproximações: filtra nós inválidos, pontua os válidos e escolhe um bom candidato, mas não explora exaustivamente todas as combinações. Essa abordagem gananciosa (greedy) pode não ser perfeita, mas é **rápida** (com complexidade polinomial) e funciona bem na prática na maioria dos cenários. Aliás, há plugins e configurações (como *Resource Bin Packing*) caso queira otimizar a utilização do cluster com estratégias mais avançadas, mas aí entra ajuste fino e possivelmente maior custo computacional no scheduling.

Evictions sob Pressão de Recursos: Mantendo o Nó Saudável

No dia a dia de um cluster, é inevitável: algum nó ficará sem memória em algum momento, ou ficará sem espaço em disco porque logs ou volumes temporários consumiram demais. Sem um gerenciamento automático, esses cenários resultariam em travamentos, degradação severa ou reinicializações caóticas de VMs. O Kubernetes aborda isso com o **Node-Pressure Eviction** – um comportamento do kubelet projetado para **recuperar recursos antes que o pior aconteça**.

Cada kubelet monitora métricas do nó local, como: - `memory.available` (memória livre ou desalocável), `nodefs.available` (espaço livre no sistema de arquivos principal do nó), `imagefs.available` (espaço livre no storage de imagens, se separado), e `pid.available` (número de PIDs disponíveis, para evitar esgotar processos no Linux).

Ele compara essas métricas a *thresholds* configurados. Existem dois tipos:

- **Hard thresholds:** limites duros que, ao serem ultrapassados, acionam evictions imediatas. Exemplo padrão: `memory.available < 100Mi` (em nós Linux), ou `nodefs.available < 10%` (de disco). Se cruzar esse limite, não tem alternativa, e o kubelet começa a matar pods quase instantaneamente para voltar abaixo do limiar.
- **Soft thresholds:** limites brandos, que precisam estar violados por um certo tempo contínuo (*grace period*) antes de agir. Por exemplo, poderíamos configurar `memory.available < 500Mi` por mais de 30 segundos como limiar soft. Assim, breves picos não acionam evictions, só situações sustentadas de pressão.

Os thresholds podem ser configurados via flags do kubelet ou arquivo de configuração, e por padrão alguns hard thresholds vêm habilitados (como o de memória mencionado). Quando múltiplos limites estão definidos, o kubelet observa todos.

O Kubernetes tenta ser “gentil” e não sair matando Pods imediatamente. Ao detectar pressão, o kubelet primeiro tenta **recuperar recursos através de limpeza interna**, removendo *pods mortos* e containers não usados (*garbage collection*), imagens antigas não utilizadas (no caso de pressão de imagefs) e libera caches se possível. Essas ações são descritas como *reclaim* e ocorrem antes da decisão de evict. Muitas vezes, só limpar imagens pode resolver um problema de disco cheio sem derrubar nenhuma aplicação.

Se mesmo após essas medidas o recurso ainda está abaixo do threshold, aí sim vem a ação mais drástica: **evictar um ou mais Pods**. Evictar, nesse contexto, significa o kubelet marcar os Pods selecionados como terminados (*Failed*) com razão de eviction e encerrá-los (com solicitação SIGTERM aos containers). É importante ressaltar que *não é um graceful shutdown* comum – o kubelet ignora o `terminationGracePeriodSeconds` dos Pods durante evictions de hard threshold, ou usa um tempo reduzido em evictions soft (padrão de 30s). Ele precisa liberar recurso rápido para salvar o nó.

A escolha de um Pod para o evict constitui um algoritmo interessante. Como não queremos tirar do ar algo importante se pudermos remover algo menos prioritário, o kubelet segue esta ordem de critérios:

1. **Pods que excedem suas requests (BestEffort ou Burstable “estourados”) vêm primeiro.** Ou seja, se um Pod não tinha request de memória (classe *BestEffort*) ou está usando mais memória do que pediu (um *Burstable* acima da request), ele é um candidato primário. Dentro desse grupo, ordena-se pela **Priority** do Pod (*PriorityClass*) e depois pelo quanto ultrapassou a request (proporcionalmente). Assim, um Pod *BestEffort* sem garantia nenhuma certamente será eliminado antes de um Pod *Guaranteed* que respeita limites.
2. **Pods *Guaranteed* ou *Burstable* dentro das requests são os últimos a sair.** Idealmente, o Kubernetes nunca evictaria Pods *Guaranteed* (aqueles cujos requests=limits) enquanto houver outros consumindo demais. Só se todos os Pods estiverem dentro do solicitado (e mesmo assim falta recurso, possivelmente por causa de daemons do sistema ou simplesmente erro de cálculo) é que ele terá de evictar algum *Guaranteed*, escolhendo o de menor prioridade primeiro.

QoS (Quality of Service) dos Pods – *Guaranteed*, *Burstable*, *BestEffort* – não é usado diretamente pelo algoritmo de eviction, mas acaba guiando indiretamente, já que *BestEffort* sempre não tem request e estará usando “acima” de 0, e *Burstable* tende a exceder requests quando causa pressão. O kubelet efetivamente olha para “usage > request” como gatilho, o que coloca *BestEffort* e alguns *Burstable* no topo da lista.

Além disso, ***PriorityClass*** é crucial. Kubernetes permite atribuir prioridades numéricas a Pods (padrão de usuário é 0, system pods críticos têm 1000000000+). O kubelet respeita isso: um Pod de baixa prioridade será sacrificado para salvar um Pod de alta prioridade se necessário. E em casos extremos onde *todos* Pods excederam requests ou todos estão dentro (e apenas daemons do sistema consumiram a memória reservada), a prioridade é o critério decisivo. Esse comportamento relaciona-se ao conceito de **preempção**: um Pod novo de alta prioridade pode *preemptar* (fazer evict) Pods de menor prioridade se não houver espaço para rodar, garantindo que workloads críticos tenham precedência.

Quando ocorre eviction, é possível que vários Pods sejam terminados de uma vez. Isso pode afetar disponibilidade da aplicação se não for bem dimensionada (por isso recomenda-se ter Pods replicados e com requests adequados para minimizar a chance de eviction). Como o Kubernetes não respeita a configuração `PodDisruptionBudget` (PDB) em evictions de pressão, mesmo que você diga “mantenha pelo menos um Pod desse Deployment rodando”, se tiver que evictar, ele evicta. Afinal, é uma emergência de nó.

Parâmetros como `eviction-max-pod-grace-period` controlam o tempo de grace geral para evictions soft. Também há o `eviction-minimum-reclaim` que podemos configurar para dizer “quando chegar no threshold, libere pelo menos X recurso”. Por exemplo, se `eviction-minimum-reclaim` para memória for 500Mi, o kubelet não vai parar após remover um Pod se a memória livre não subiu pelo menos 500Mi acima do threshold – isso evita ficar *evictando um por um* vários Pods em sequência em um cenário de pressão contínua (oscilações rápidas). Sem isso, poderia ocorrer um cenário de *oscillatory eviction*: o nó quase sem memória, mata um Pod leve, libera só um tiquinho, continua abaixo do threshold, mata outro, e assim por diante – um efeito dominó possivelmente exagerado. Com mínimo *reclaim*, ele tenta garantir que a ação resolva a situação com folga, evitando este processo de remoção de pods.

A lógica é semelhante para espaço em disco (`nodefs` e `imagefs`). O kubelet tenta limpar imagens não usadas primeiro. Se ainda faltar espaço, ele escolhe Pods para remover. Nesse caso, não há “requests” de disco e QoS para usar, então a ordem se baseia principalmente em prioridade e em quem está usando mais espaço. Se `nodefs` (disco geral) está cheio, ele ordena Pods por uso de volumes locais + logs (tudo que escreve no filesystem do nó). Por outro lado, se for `imagefs` (camada de imagens/overlay), ordena por uso da camada de escrita dos containers. Por fim, Inodes e PIDs também não têm requests, e o kubelet simplesmente remove Pods de menor prioridade até voltar ao normal.

Uma coisa interessante: **eviction não é igual a OOM kill de container**. Se um container individual alocado excede sua memória *limite*, o kernel OOM killer pode matá-lo, e o kubelet apenas reporta que aquele container terminou por OOM. Isso não é o mecanismo de eviction – é o cgroup aplicando limites por processo. Evictions entram em jogo quando o *conjunto* de Pods está estressando o nó em geral. Quando

um Pod é OOM-killed, ele pode reiniciar automaticamente (se tiver a configuração *restartPolicy* definida) dentro do mesmo nó. Já uma eviction retira o Pod do nó (como se fosse um *expurgo* do nó). Portanto, Kubernetes tem dois níveis de defesa: por Pod (cgroups, OOM kill se passar do limit) e por Nó (eviction se mesmo dentro dos limites o nó saturar devido a muitos Pods somados ou Pods mal configurados).



MERCADO, CASES E TENDÊNCIAS

Para fechar a discussão da desta aula, vamos olhar para alguns exemplos do mundo real onde esses conceitos de agendamento avançado e comportamento de nós se mostraram cruciais – seja em histórias de sucesso, seja em aprendizados por falhas.

Isolamento de Cargas Sensíveis (Segurança)

Em 2022, uma equipe de segurança realizando *pentest* em clusters descobriu que se conseguissem comprometer um nó Kubernetes, podiam usar a conta de serviço de qualquer Pod ali rodando para escalar privilégios (roubando tokens). Uma das recomendações resultantes foi **isolar workloads sensíveis em nós dedicados** usando taints/tolerations e affinities, de forma que, mesmo que um nó de menor confiança seja atacado, os pods críticos não estejam misturados nele.

Por exemplo, colocar toda a carga de clientes VIP em nós separados (taintados como `vip-only:NoSchedule`) reduz o *blast radius*. Essa prática de “workload isolation” é adotada em setores como finanças e governo, onde diferentes classificações de dados vivem em diferentes nós físicos ou até diferentes clusters. O blog da Padok/Theodo deu um guia de como combinar taints e node affinity para este isolamento robusto, enfatizando exatamente que usar só um ou outro mecanismo deixa brechas, mas juntos eles garantem que workloads sensíveis fiquem nos seus “santuários” e nada mais entra, nem mesmo por descuido.

Maximizando Economia com Nós Spot (Custo)

Uma grande empresa farmacêutica obteve destaque em um *case study* por economizar **76% dos custos** de computação em workloads de Machine Learning usando instâncias spot/preemptíveis e Kubernetes. Como conseguiram?

Eles configuraram um nó group de instâncias spot com um taint como `preemptible=true:NoSchedule`. Os jobs de ML de longa duração receberam tolerations para esse taint, indicando que podiam rodar em spot nodes (mais baratos, porém podem sumir a qualquer momento). Além disso, definiram node affinity para preferir esses nós spot.

O resultado: os Pods de ML quase sempre iam para nós spot tolerados, barateando enormemente a execução. Quando a nuvem tirava uma instância spot do ar, os Pods eram evictados, mas como eram jobs tolerantes e reiniciáveis, o controlador os reprogramava em outra instância (a empresa automatizou reaproveitar e substituir nós spot perdidos). Esse caso mostra o poder de combinar *tolerations* + *affinity* + *automação customizada* para reduzir custos – o cluster “rodou com os socos” da volatilidade das instâncias, tolerando interrupções, mas sempre voltando a treinar os modelos.

A Cast AI, que divulgou o caso, sugere fortemente esse padrão para quem quer cortar custos em até 70% sem perder (muito) em confiabilidade, usando Kubernetes para gerenciar a fluidez dos recursos de forma inteligente.

Eviction Storm – Lição Aprendida

Um exemplo de fracasso evitado veio de uma startup de SaaS que, certa noite, viu centenas de Pods reiniciando simultaneamente em seu cluster de produção – causando instabilidade generalizada do serviço. O diagnóstico posterior revelou que um *daemon* de log mal configurado começou a gerar arquivos enormes em várias máquinas, enchendo o disco (condição de DiskPressure).

O kubelet desses nós acionou evictions, mas a configuração padrão não tinha `eviction-minimum-reclaim` bem ajustado. Resultado: assim que liberava um pouco de espaço matando 1 ou 2 Pods, algum processo de log voltava a escrever e logo em seguida outro Pod era evictado, em efeito dominó. Quase 50 Pods foram derrubados em questão de minutos.

Após esse incidente, a equipe implementou **limites de retenção de logs (log rotation)** – para evitar encher disco – e ajustou os thresholds para ter mais folga. Além disso, adicionaram **monitoração pró-ativa**: agora um alerta dispara se um nó passa de 85% de uso de disco ou memória, antes mesmo de chegar nos 95%+ que causam eviction.

Assim, operadores podem drenar ou investigar o nó manualmente. Essa história (não publicamente referenciada com nomes por questões óbvias) ecoa a recomendação de muitos especialistas: *evictions são o último recurso*. Empresas maduras tentam evitar chegar a esse ponto com observability e capacidade extra. Ainda assim, é melhor ter o mecanismo de eviction do que o node simplesmente

travar. Como disse um engenheiro, “**prefiro alguns Pods reiniciados à máquina inteira congelada**”.

Taints e QoS no Kubernetes da Apple

Uma curiosidade de caso de uso vem de relatos da KubeCon: a Apple comentou sobre o uso intenso de taints/tolerations combinados com *PriorityClasses* e QoS para garantir que cargas em tempo real tivessem nós exclusivos. Eles nos marcaram com taints do tipo `realtime=true:NoSchedule` para isolar aplicações de áudio/vídeo de baixa latência, toleradas apenas por esses workloads especiais. Além disso, definiram esses Pods como Guaranteed (com `requests=limits`) e Priority muito alta.

Assim, nenhum outro Pod iria para esses nós (graças aos taints) e mesmo se o nó ficasse pressionado, aqueles Pods seriam os últimos a serem evictados. Esse design mostra como Kubernetes pode ser afinado para casos muito específicos – praticamente criando “classes” de nós e workloads. Não é trivial nem recomendado para todos, mas em ambientes exigentes faz a diferença entre cumprir ou não um SLA rigoroso.

Escalabilidade do Scheduler (Borg e Kubernetes)

Em termos de capacidade técnica, vale mencionar que o agendador do Kubernetes herdou lições do Google Borg (sistema interno que inspirou o K8s). O Borg rodava milhões de contêineres e precisava decidir colocação rapidamente; para isso ele também usava uma mistura de *heurísticas glutton* (gulosa) com backtracking limitado.

Pesquisas acadêmicas e o *Borg paper* indicam que rodar um algoritmo NP-hard otimizado exatamente estava fora de questão – eles buscavam boas soluções em milissegundos. Kubernetes segue essa linha: prioriza rapidez sobre ótima utilização global. Isso é perceptível pois às vezes vemos clusters com algum desalinhamento (nós com 70% de uso e outros com 50%) porque o scheduler faz decisões locais sem reembaralhar constantemente.

Porém, a simplicidade do algoritmo traz robustez e facilidade de depurar. Empresas que precisam mais otimização recorrem a add-ons (como o Descheduler, Cluster Autoscaler, entre outros mecanismos) para lapidar a alocação a posteriori.

O QUE VOCÊ VIU NESTA AULA?

Vimos que a gestão avançada de agendamento e nós no Kubernetes é rica em possibilidades e aprendizados. Casos de sucesso mostram que, usando bem essas ferramentas, podemos obter **clusters mais seguros, estáveis e eficientes**, isolando o que deve ser isolado, economizando onde dá para economizar e reagindo de forma automática a imprevistos de recurso.

Por outro lado, os casos problemáticos nos lembram de configurar essas funcionalidades com sabedoria: thresholds mal calibrados ou esquecimento de alguma toleration podem causar comportamentos indesejados (downtime ou falta de isolamento).

Para um profissional de operações de computação em nuvem, dominar esses conceitos é fundamental para **orquestrar contêineres em escala com confiabilidade**, e faz a diferença entre um cluster “mais ou menos” e um cluster verdadeiramente de nível enterprise. **Bons deploys!**

REFERÊNCIAS

Kubernetes. “**API-initiated Eviction.**” Kubernetes Documentation. 2025. Disponível em: <https://kubernetes.io/docs/concepts/scheduling-eviction/api-eviction/>. Acesso em: 04 set. 2025.

Kubernetes. “**Assigning Pods to Nodes.**” Kubernetes Documentation. 2025. Disponível em: <https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/>. Acesso em: 04 set. 2025.

Kubernetes. “**kubectl taint.**” Kubernetes Documentation. 2025. Disponível em: https://kubernetes.io/docs/reference/kubectl/generated/kubectl_taint/. Acesso em: 04 set. 2025.

Kubernetes. “**Node-pressure Eviction.**” Kubernetes Documentation. 2025. Disponível em: <https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/>. Acesso em: 04 set. 2025.

Kubernetes. “**Pod Priority and Preemption.**” Kubernetes Documentation. 2025. Disponível em: <https://kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/>. Acesso em: 04 set. 2025.

Kubernetes. “**Pod Topology Spread Constraints.**” Kubernetes Documentation. 2025. Disponível em: <https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/>. Acesso em: 04 set. 2025.

Kubernetes. “**Scheduling, Preemption and Eviction.**” Kubernetes Documentation. 2024. Disponível em: <https://kubernetes.io/docs/concepts/scheduling-eviction/>. Acesso em: 04 set. 2025.

Kubernetes. “**Taints and Tolerations.**” Kubernetes Documentation. 2025. Disponível em: <https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/>. Acesso em: 04 set. 2025.

Kubernetes. “**Well-Known Labels, Annotations and Taints.**” Kubernetes Documentation. 2025. Disponível em: <https://kubernetes.io/docs/reference/labels-annotations-taints/>. Acesso em: 04 set. 2025.

Lewis, Ian, e David Oppenheimer. “**Advanced Scheduling in Kubernetes.**” Kubernetes Blog. 2017. Disponível em: <https://kubernetes.io/blog/2017/03/advanced-scheduling-in-kubernetes/>. Acesso em: 04 set. 2025.

EMSE

PALAVRAS-CHAVE

Kubernetes. Agendamento. Controle de Nós.

EMSE



POSTECH